Transformations in Signal Processing

Review the following matrix transformations, $T : \mathbf{R}^n \to \mathbf{R}^n$ explained in the handout and write computer functions to return the transformed vector. For coding assignments, we always write efficient codes that produce the output as we designed. The codes may not necessarily follow the matrix structure we conceptualize in math. Almost all of these transformation can be computed by using commands in NumPy.

1. Reverser: Return $T(x) = (x_n, x_{n-1}, \ldots, x_2, x_1)$ in `reverser(x)`.
2. Downsampling: Given $x \in \mathbf{R}^n$ and an integer s where $1 \leq s \leq n$, return $y = T(x)$ by selecting every s-th entry of x. That is, $y = (x_s, x_{2s}, x_{3s}, \ldots, \lfloor n/s \rfloor)$. Write `downsampling(x,s)`
3. Forward and backward shift: Forward shift returns $y = (x_2, x_3, \ldots, x_n, x_1)$, while backward shift returns $y = (x_n, x_1, x_2, \ldots, x_{n-1})$. Write `myshift(x, opt)` where `opt` can be `f` or `b` as forward or backward shift and the default is forward.
4. Differencing: $T : \mathbf{R}^n \to \mathbf{R}^{n-1}$ that gives

$$T(x) = \begin{bmatrix} x_2 - x_1 \\ x_3 - x_2 \\ \vdots \\ x_n - x_{n-1} \end{bmatrix}$$

   Write `mydiff(x)`.
5. Cumulative sum: Write `mycumsum(x)`.

$$T(x) = \begin{bmatrix} x_1 \\ x_1 + x_2 \\ \vdots \\ x_1 + x_2 + \cdots + x_n \end{bmatrix}$$

Input Format

- a vector $x \in \mathbf{R}^n$

Output Format

- $y \in \mathbf{R}^n$: The transformed output.

Implementation

Goal: Fill in the following function:

```
import numpy as np

def reverser(x):
    ...
    return

def downsampling(x, s):
    ...
    return

def myshift(x, opt='f'):
    ...
    return

def mydiff(x):
    ...
    return

def mycumsum(x):
    ...
    return
```